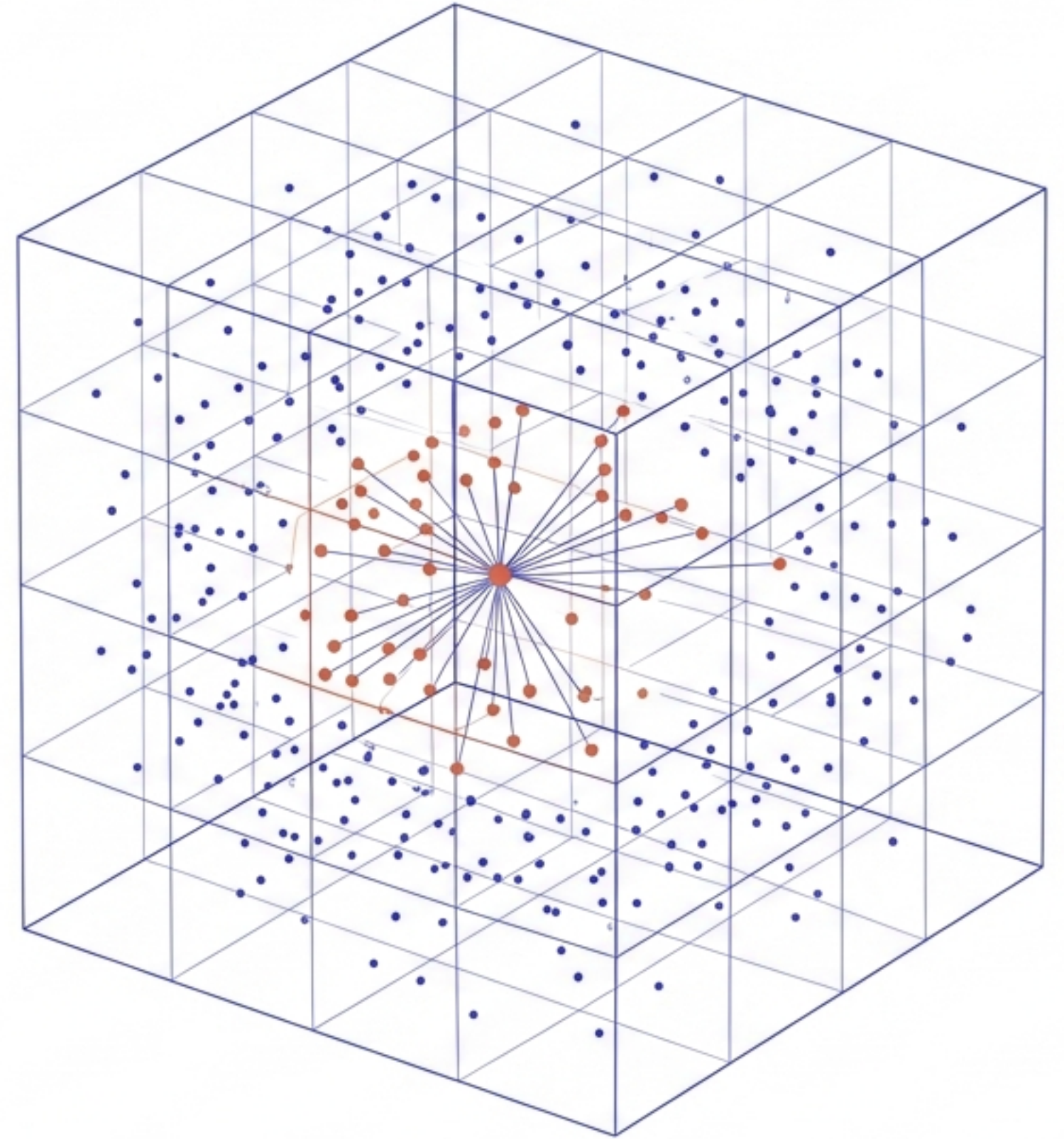


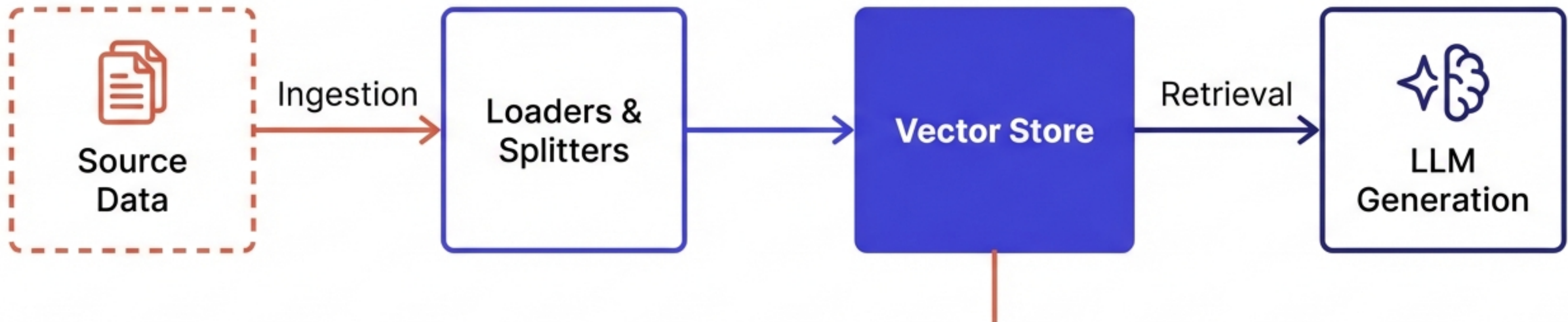
Vector Stores in LangChain

The Backbone of Semantic Search and RAG Applications

From simple keyword matching to understanding "meaning." This deck explores the architecture of **Vector Stores**, why traditional databases fail at AI tasks, and how to implement them using LangChain.



The Bridge Between Data and LLMs

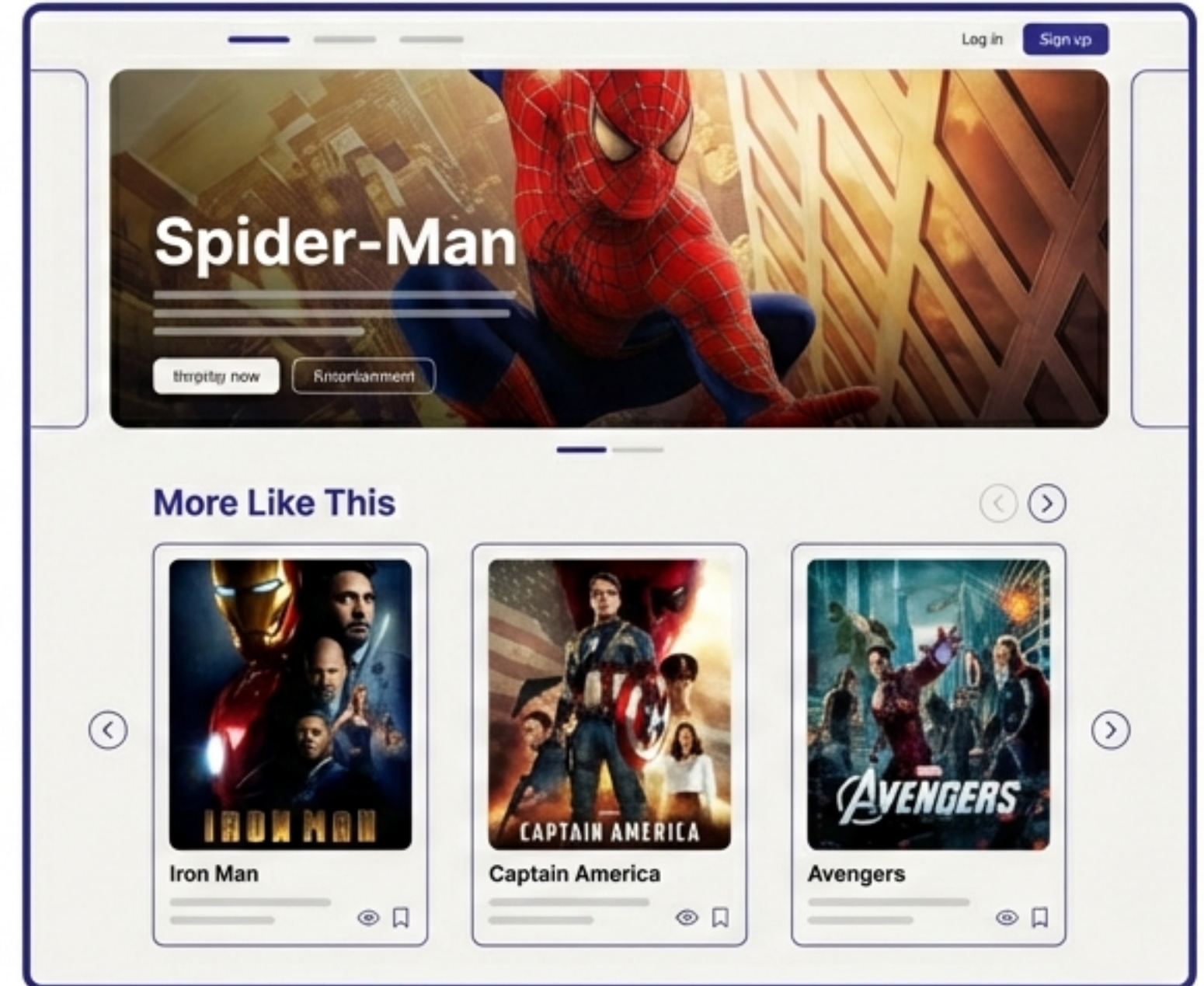


The 'Long-Term Memory' where semantic data is stored, indexed, and retrieved for the LLM.

The Use Case: Building an Intelligent Movie Recommender

Imagine building a movie catalog. A user is viewing *Spider-Man*. To keep them engaged, we need to recommend similar films (e.g., *Iron Man*, *Captain America*) at the bottom of the page.

Goal: Increase time-on-site and ad revenue by surfacing relevant content.



The Trap of Keyword Matching



Failure Case 1: The False Positive

User watches: "My Name is Khan" (Serious Drama)

Algorithm Recommends: "Kabhi Alvida Naa Kehna"
(Relationship Drama)

Why it failed:

Shared keywords (Actor: Shah Rukh Khan, Director: Karan Johar) but totally different moods.



Failure Case 2: The Missed Connection

User watches: "Taare Zameen Par"

Algorithm Ignores: "A Beautiful Mind"

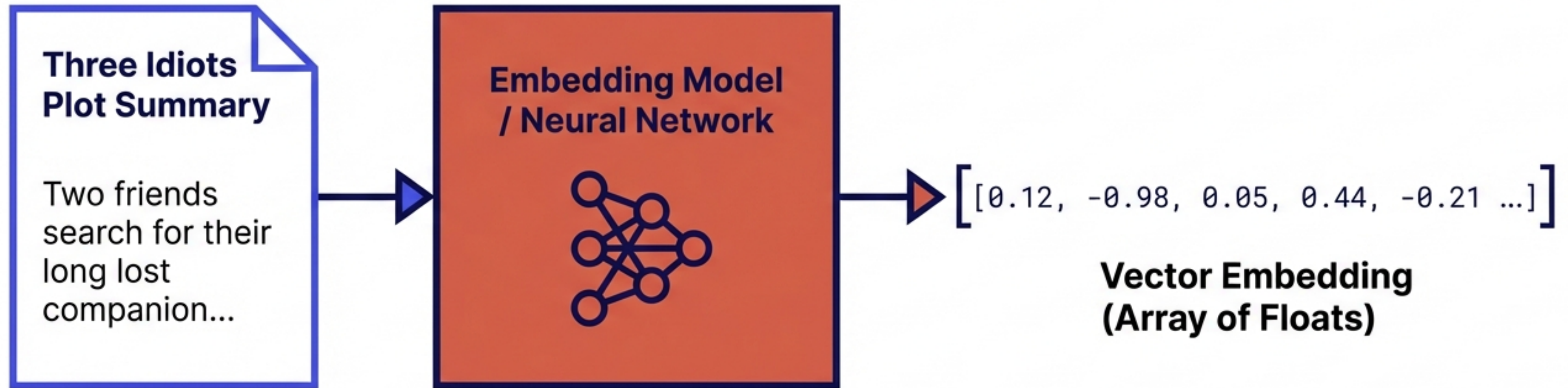
Why it failed:

Zero shared keywords (Different actors, language, era) despite identical themes of struggle and brilliance.

Insight: Keywords capture metadata. They do not capture the *soul* or *story* of the content.

The Solution: Capturing 'Meaning' with Embeddings

Instead of matching Director names, we compare the **Movie Plot**.



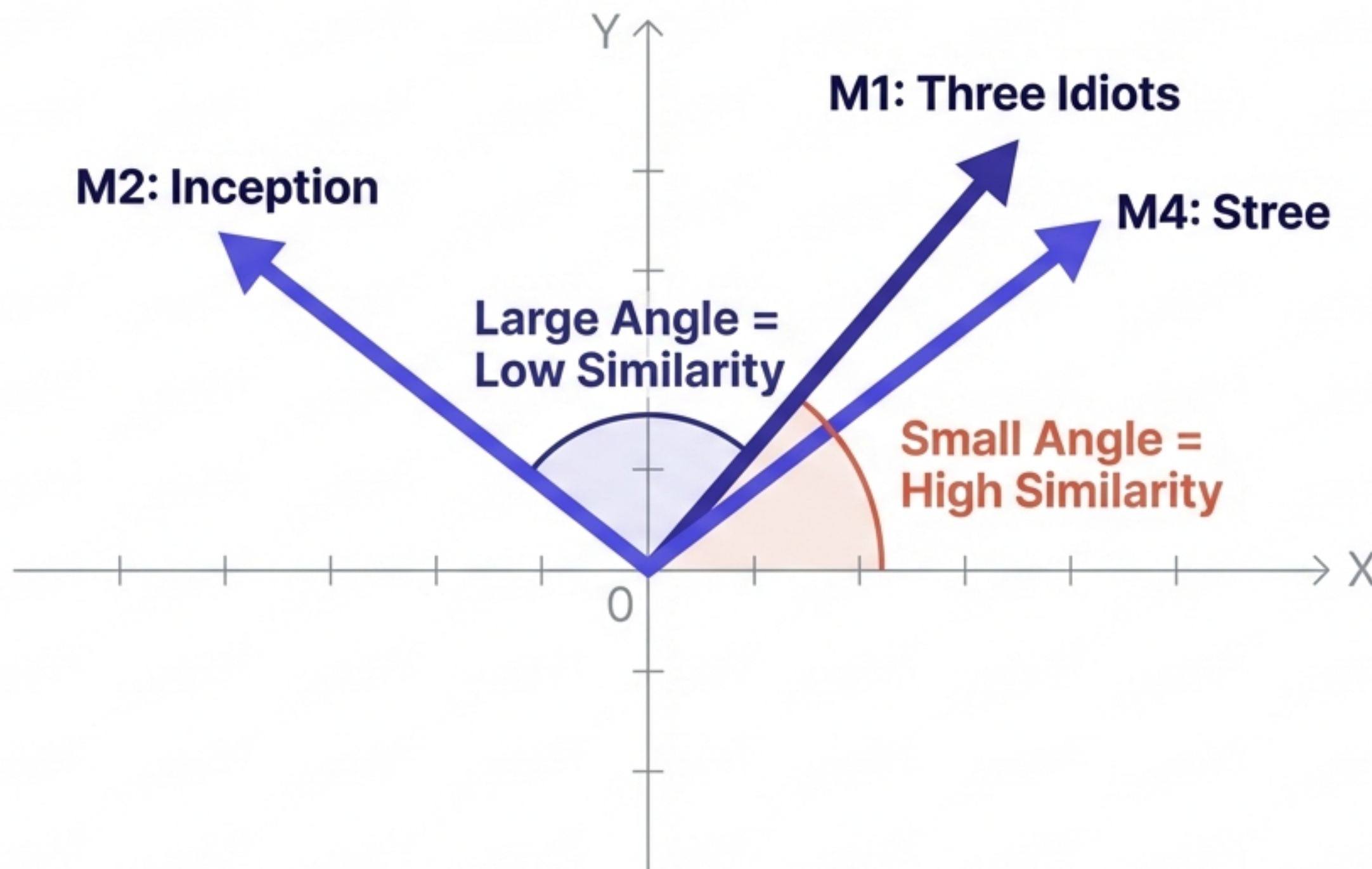
Embeddings transform semantic meaning into fixed-size lists of numbers. The machine understands the story as a mathematical coordinate.

How Machines 'See' Similarity

Cosine Similarity measures the angle between vectors.

- **Distance** = Difference
- **Angle** = Similarity

The model recognizes **Stree** is **closer to *Three Idiots*** (comedy/drama) than to a sci-fi thriller like ***Inception***.



Why Traditional Databases Fail Here



Generation Scale

Inter Tight

Creating embeddings for millions of items is resource-intensive.



Storage Inefficiency

Inter Tight

Relational DBs (MySQL/Oracle) are not optimized for storing massive arrays of floating-point numbers.

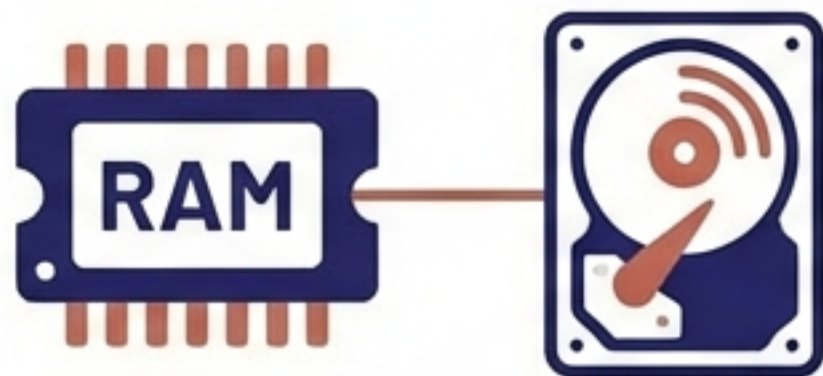


The Speed Bottleneck ($O(N)$)

Comparing 1 query against 1,000,000 vectors row-by-row is too slow for real-time applications.

Defining the Vector Store

A system specialized to Store, Index, and Retrieve numerical vectors.



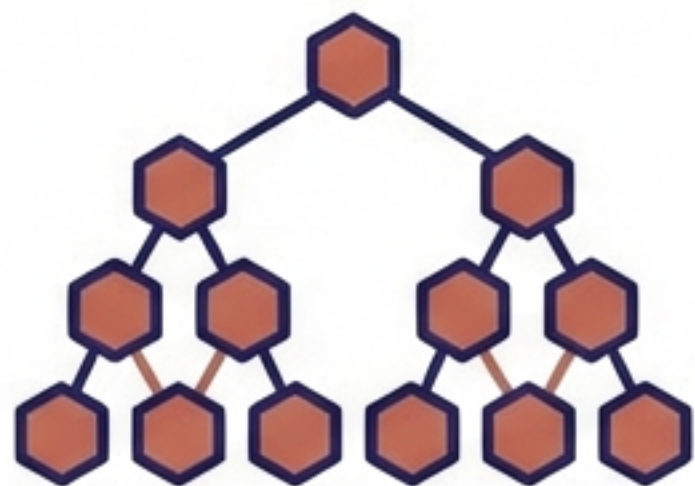
Storage

Memory (Fast) vs.
Disk (Persistent).



Similarity Search

Native algorithms to
calculate nearest
neighbors.



Indexing

The architecture that
solves the speed
problem (HNSW, IVF).

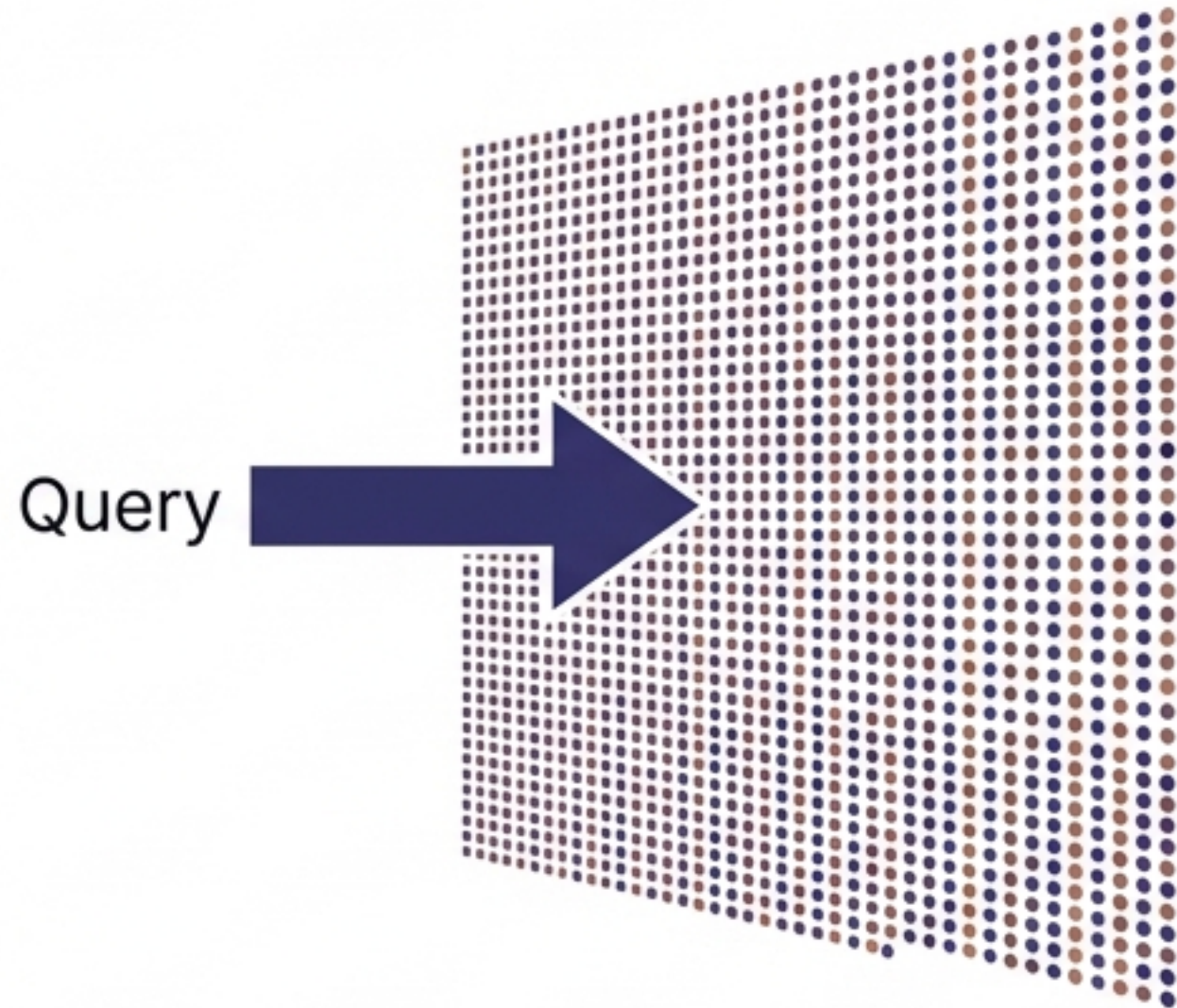


CRUD Operations

Create, Read, Update,
Delete vector records.

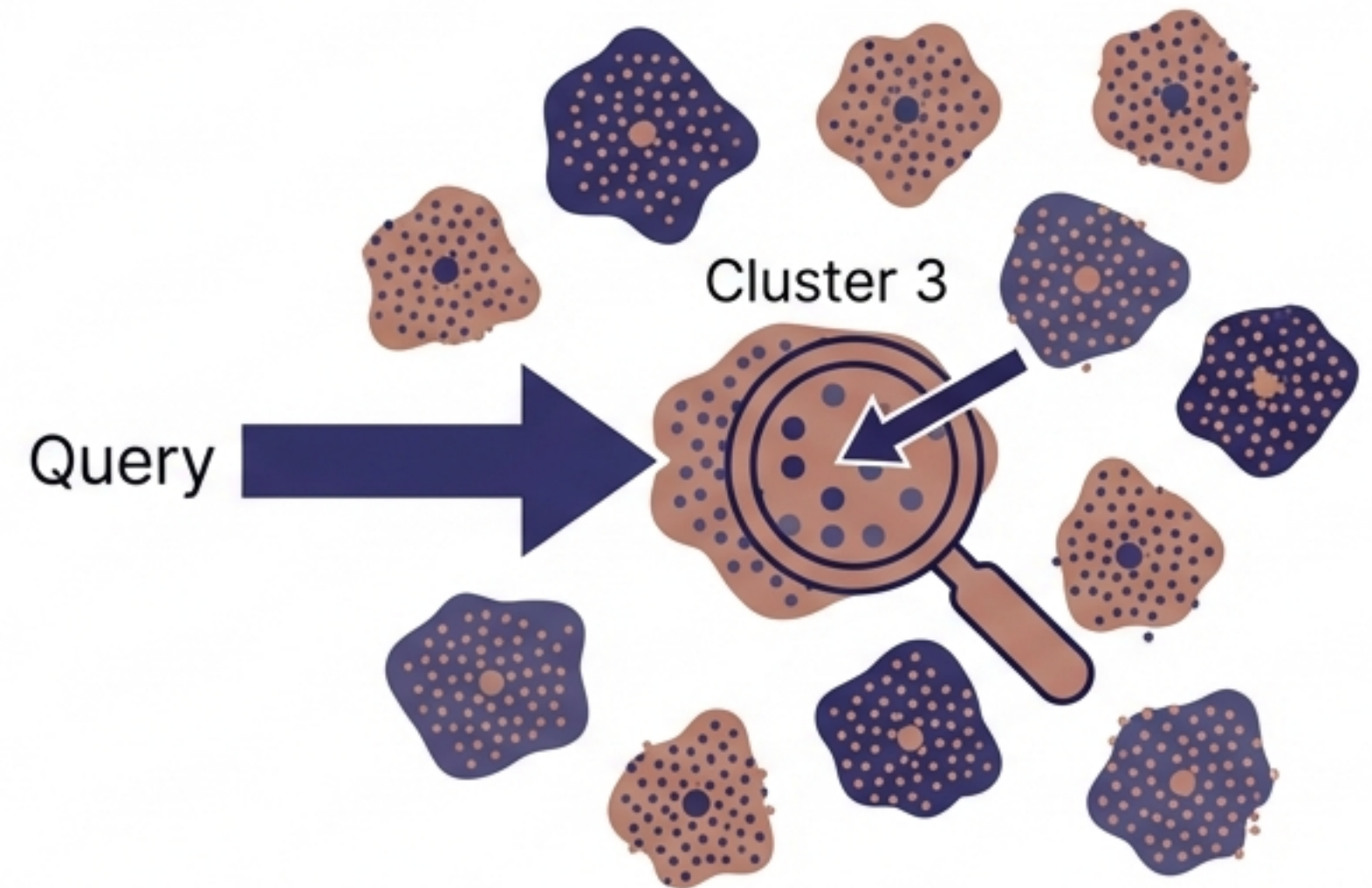
Solving the Speed Limit via Indexing

The Slow Way (Linear Search)



Checking 1 vs 1M = $O(N)$

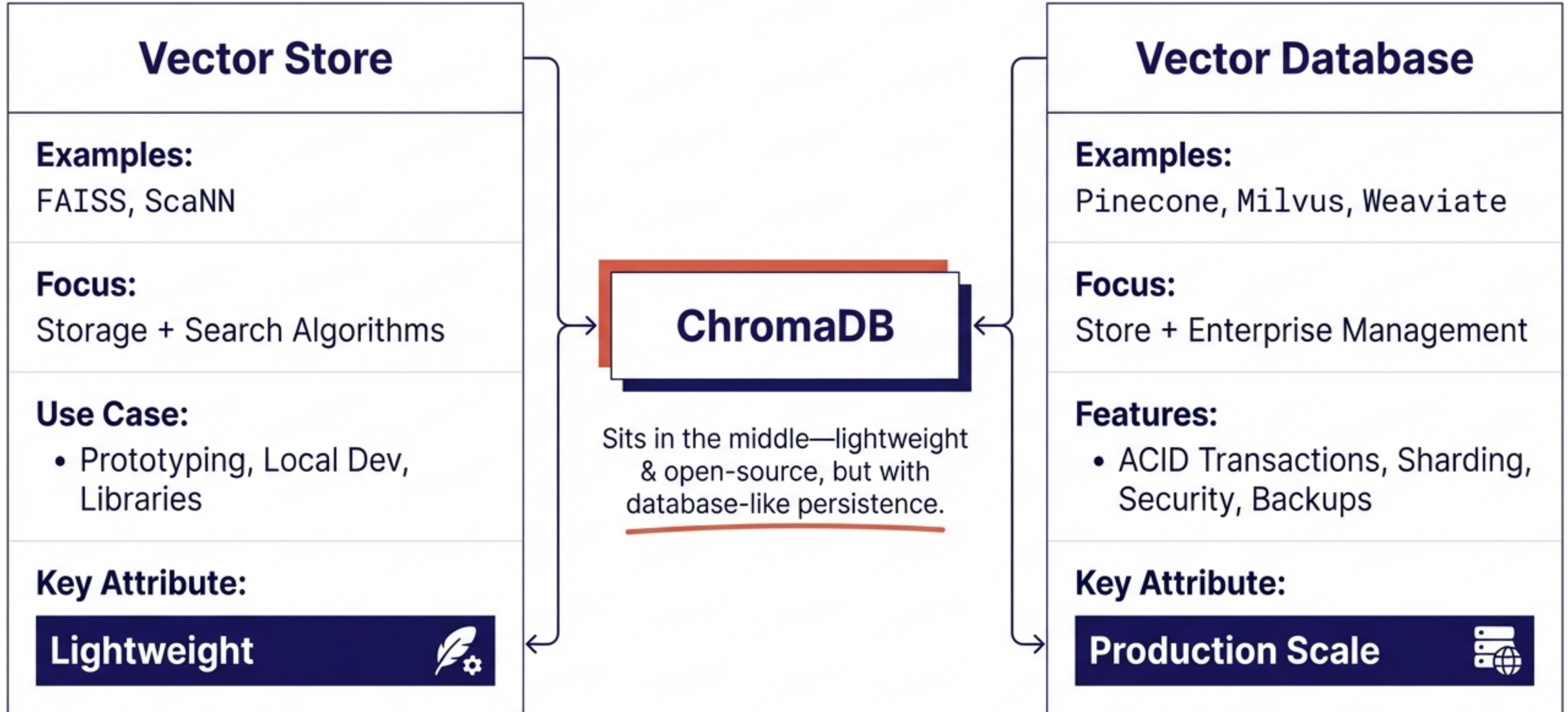
The Vector Store Way (Clustering)



1. Cluster 1M vectors into 10 groups.
2. Compare query to centroids.
3. Search only the best cluster (~100k vectors).

Result: Drastically reduced computations. This is Approximate Nearest Neighbor (ANN) search.

Vector Store vs. Vector Database



The LangChain Advantage

LangChain Unified Interface

```
.similarity_search()  
.add_documents()
```

Switch from local dev
(Chroma) to production
(Pinecone) with **zero
code logic changes**.
The interface remains
the same.



Chroma



Pinecone



FAISS



Qdrant

Implementation: Setup & Ingestion

Context: Building a search tool for Cricket players.

1. Create Documents

```
docs = [  
    Document(page_content="Virat Kohli...", metadata={"team": "RCB"}),  
    Document(page_content="MS Dhoni...", metadata={"team": "CSK"}),  
    Document(page_content="Rohit Sharma...", metadata={"team": "MI"}),  
    ...  
]
```

2. Initialize Vector Store (Chroma)

```
db = Chroma.from_documents( ←  
    documents=docs,  
    embedding=OpenAIEmbeddings(),  
    persist_directory="./chroma_db"  
)
```

This function automatically tokenizes text, generates embeddings via OpenAI, and indexes them locally.

Implementation: Semantic Search in Action

```
query = "Who is a bowler?"  
docs = db.similarity_search(query, k=2)
```

The Results

Jasprit Bumrah

...known for his yorkers and death bowling...

Rank 1

Ravindra Jadeja

...an all-rounder who bowls left-arm spin...

Rank 2

The model understands 'Bowler' relates to Bumrah and Jadeja without explicit keyword tags.

Advanced Operations: Filtering & Management

Metadata Filtering

```
db.similarity_search(query, filter={"team": "CSK"})
```

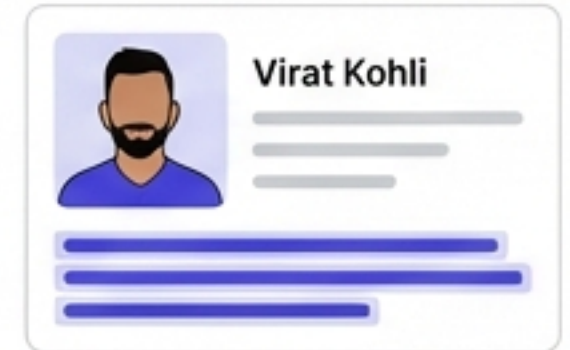
Returns only
Dhoni & Jadeja.



Update Document

```
db.update_document(id, new_document_data)
```

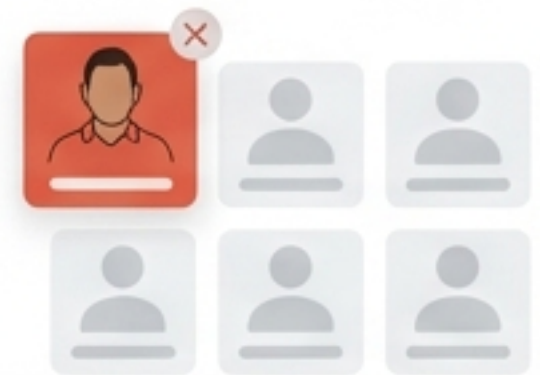
Updating **Virat Kohli**'s profile text.



Delete Document

```
db.delete([id])
```

Removing a retired
player from the index.



The Foundation of Intelligent Apps

Key Takeaways

- **From Keywords to Meaning:** Vector stores match the soul of the content, not just the metadata.
- **Speed at Scale:** Indexing strategies (like clustering) make searching millions of vectors viable.
- **LangChain Flexibility:** One unified interface allows you to develop once and deploy to any backend.

Next Steps



Connecting this Retriever to an LLM Generator.